TOBB ETÜ
Ekonomi ve Teknoloji Üniversitesi

TOBB EKONOMİ VE TEKNOLOJİ ÜNİVERSİTESİ

Bilgisayar / Yapay Zekâ Mühendisliği

BİL 495 — YAP 495

Analiz Raporu

Proje Adı	Akıllı Trafik Akış Simülasyonu ve Yol Tasarım Karar Destek Sistemi
Takım Üyeleri	Beril Aydın 231101002 Kaan Baykal 221101028 Duygu Akman 221101035 Yankı Yağız Ozan 231101059
Danışman	
Akademik Dönem	2025-2026 Yaz Dönemi
Teslim Tarihi	12.07.2026

İÇİNDEKİLER

1. GİRİŞ..... 2
1.1 Amaç..... 2
1.2 Okuyucu Kitlesi..... 3
2. MEVCUT SİSTEM..... 3
2.1. Ticari ve Kapalı Kaynaklı Kurumsal Yazılımlar..... 3
2.2. Açık Kaynaklı Akademik Simülasyon Motorları..... 4
2.3. Mevcut Sistemlerin Eksikliği ve İhtiyaç Analizi..... 4
3. ÖNERİLEN SİSTEM..... 4
3.1 Overview..... 4
3.2 Fonksiyonel Gereksinimler..... 5
3.3 Fonksiyonel Olmayan Gereksinimler..... 8
3.4 Pseudo Gereksinimler (Teknoloji/Platform Kısıtları)..... 10
4. SİSTEM MODELLERİ..... 11
4.1. Senaryolar (Scenarios)..... 11
4.2 Use Case Model..... 13
4.2.1. Aktörler..... 13
4.2.2 Use Case Diyagramı..... 14
4.2.3. Kullanım Durumları..... 14
4.3. Nesne ve Sınıf Modeli..... 17
4.3.2. Sınıf Tanımları ve Sorumluluklar..... 18
4.4 Dinamik Modeller..... 20
4.4.1. Aktivite Diyagramı : Senaryo Yönetim ve Doğrulama Akışı..... 20
4.4.2. Ardışık Diyagram : Simülasyon İstek Döngüsü..... 22
4.5. Kullanıcı Arayüzü — Navigasyon ve Mock-up Hiyerarşisi..... 23
5. SÖZLÜK..... 28
6. KAYNAKLAR..... 30
7. ARAÇLAR VE METODOLOJİ..... 30

1. GİRİŞ

1.1 Amaç

Bu analiz raporunun temel amacı, Akıllı Trafik Akış Simülasyonu ve Yol Tasarım Karar Destek Sistemi projesinin mantıksal, davranışsal ve operasyonel modelini uçtan uca ve şeffaf bir biçimde ortaya koymaktır. Karmaşık kentsel trafik sorunlarına çözüm üretmeyi hedefleyen bu sistem, donanım ve lisans bağımlı mevcut ticari araçların aksine; erişilebilir, web tabanlı ve hızlı senaryo testine olanak tanıyan bir "What-if?" karar destek mekanizması olarak tasarlanmıştır.

Bu rapor, sistemin teknik olarak nasıl kodlanacağından çok, temel işlevlerinin neler olduğuna odaklanmaktadır. Tasarlanan sistemin çalışacağı kısıtlar, kabul edeceği girdiler, üreteceği çıktılar ve faydalanacağı dış servisler net bir şekilde sınırlandırılmıştır. Böylece projenin kapsamı açıkça çizilerek, hem akademik değerlendiriciler hem de potansiyel son kullanıcılar (şehir planlamacıları, ulaştırma mühendisleri ve yerel yönetimler) için hedeflenen fonksiyonlar somutlaştırılmıştır.

1.2 Okuyucu Kitlesi

Bu doküman projenin değerlendirme, kullanım ve olası geliştirme süreçlerinde yer alacak farklı paydaşlar için hazırlanmıştır. Rapordaki teknik detaylar ve sistem kısıtları, aşağıdaki okuyucu gruplarının beklentileri göz önünde bulundurularak yapılandırılmıştır:

- **Akademik Değerlendiriciler ve Proje Danışmanları:** Geliştirilen sistemin araştırma metodolojisini, tasarım kararlarını ve sistem mimarisini değerlendirecek olan birincil okuyucu kitlesidir. Bu kitle, projenin akademik standartlara uygunluğunu denetlemek ve öngörülen hedeflerin ne ölçüde karşılandığını analiz etmek için bu dokümanı referans alacaktır.
- **Şehir Planlamacıları ve Ulaştırma Mühendisleri:** Sistemin hedeflediği potansiyel son kullanıcılardır. Tasarlanan mimarinin yol kapatma, şerit ekleme veya tek yönlü yol dönüşümü gibi düşük maliyetli kentsel müdahaleleri nasıl modellediğini anlamak için bu belgeyi inceleyebilirler. Ayrıca, simülasyon çıktısı olarak üretilen trafik yoğunluğu, bekleme süresi ve kuyruk uzunluğu gibi metriklerin karar destek süreçlerine nasıl entegre edildiğini görmek için bu rapora başvurabilirler.
- **Yazılım Geliştiriciler ve Araştırmacılar:** Kurulan asenkron worker tabanlı altyapıyı ve dosya tabanlı alt süreç mimarisini incelemek isteyen teknik kitledir. Bu belge, Yüksek Seviye (HLD) ve Düşük Seviye (LLD) mimari tasarımlara, API kontratlarına ve kodlama fazına geçmeden önceki temel kılavuzdur. Geliştiriciler; SUMO simülasyon motorunun sisteme nasıl entegre edileceğini, FastAPI ve React arasındaki asenkron veri akışını, job worker'ların sınırlarını ve domain modelinin sınıf yapılarını bu rapor üzerinden anlayarak teknik iş paylaşımını gerçekleştirebilirler.

2. MEVCUT SİSTEM

Günümüzde kentsel ulaşım planlaması, trafik sıkışıklığının tespiti ve altyapı müdahalelerinin test edilmesi süreçlerinde kullanılan yazılım ve sistemler incelendiğinde, sektörün belirgin bir şekilde iki ana kutba ayrıldığı görülmektedir. Mevcut ekosistemde yer alan çözümler, hitap ettikleri kullanıcı profilleri ve teknolojik altyapıları gereği çeşitli avantajlar sunarken, karar destek süreçlerini yavaşlatan ciddi darboğazlar da barındırmaktadır.

2.1. Ticari ve Kapalı Kaynaklı Kurumsal Yazılımlar

Sektör standardı olarak kabul edilen PTV Vissim, Aimsun ve TransModeler gibi kapalı kaynaklı ticari yazılımlar[1], yüksek doğruluk oranlarına sahip mikroskobik ve makroskobik simülasyon araçlarıdır. Ancak bu sistemlerin temel kısıtları şunlardır:

Yüksek Lisans ve Kurulum Maliyetleri: Yıllık lisans ücretleri yüz binlerce doları bulabilmektedir. Bu durum, özellikle kısıtlı bütçeye sahip orta ve küçük ölçekli yerel yönetimlerin veya bağımsız araştırmacıların bu teknolojilere erişimini imkansız kılmaktadır.

Donanım Bağımlılığı ve Kurulum Zorluğu: Genellikle "thick client" (kalın istemci) mimarisiyle çalışan masaüstü uygulamalarıdır. Yüksek işlem gücüne sahip özel iş istasyonları gerektirirler ve bulut/web tabanlı esnek bir erişim sunmazlar.

Dik Öğrenme Eğrisi: Arayüzleri oldukça karmaşıktır. Bu yazılımları kullanabilmek, senaryo üretebilmek ve sonuçları yorumlayabilmek için haftalar süren özel sertifikasyon eğitimleri almış uzmanlara ihtiyaç duyulmaktadır.

2.2. Açık Kaynaklı Akademik Simülasyon Motorları

Ticari yazılımlara alternatif olarak, akademi ve araştırma dünyasında geniş kabul gören Eclipse SUMO[2] ve MATSim[3] gibi açık kaynaklı sistemler bulunmaktadır. Geliştirilecek projenin de çekirdeğinde yer alacak olan bu araçlar, trafik akış fiziğini modellemede son derece güçlü olmalarına rağmen doğrudan bir "son kullanıcı ürünü" değildirler:

Arayüz Eksikliği ve Teknik Karmaşa: Bu sistemler salt birer simülasyon "motoru" olarak tasarlanmıştır. Dahili grafik arayüzleri senaryo tasarlamaktan ziyade, önceden kodlanmış bir simülasyonu izlemek içindir.

Kullanım Zorluğu (Kodlama Gereksinimi): Şehir plancısı veya karar verici pozisyonundaki bir kullanıcının yeni bir "Ne Olurdu?" senaryosu test edebilmesi için; ağ topolojisini barındıran devasa XML dosyalarını manuel olarak düzenlemesi, terminal üzerinden komut satırı araçlarını çalıştırması ve süreçleri yönetmek için Python betikleri yazması gerekmektedir.

Ham Çıktıların İşlenmesi: Simülasyon tamamlandığında sonuçlar görsel bir rapor yerine gigabaytlarca büyüklükte ham log dosyaları olarak üretilir. Bu verilerin anlamlı makroskobik metriklere dönüştürülmesi ciddi bir veri mühendisliği eforu gerektirir.

2.3. Mevcut Sistemlerin Eksikliği ve İhtiyaç Analizi

Mevcut durumda, kentsel altyapı kararları alan yöneticiler ve planlamacılar için kapalı kaynaklı ticari yazılımlar ile açık kaynaklı akademik simülasyon motorları arasında pratik ve erişilebilir bir ara çözüm bulunmamaktadır. Bir şeridin trafiğe kapatılması veya bir kavşağa katılım şeridi eklenmesi gibi altyapı müdahalelerinin etkilerini hızlıca analiz etmek isteyen karar vericiler, operasyonel zorluklarla karşılaşmaktadır. Bu analiz sürecinin yürütülebilmesi için ya yüksek maliyetli ve uzun süreli ticari danışmanlık hizmetlerine başvurulmakta ya da teknik ekiplerin karmaşık altyapı dosyalarını manuel olarak yapılandırması için uzun işlem süreleri beklenmek zorundadır.

3. ÖNERİLEN SİSTEM

3.1 Overview

Önerilen sistem; güçlü ancak karmaşık olan SUMO mikroskobik simülasyon motorunu bir "kara kutu" olarak arka planda çalıştıran, kullanıcı etkileşimini ise modern, web tabanlı ve harita odaklı (React tabanlı) bir arayüze taşıyan bütünleşik bir Karar Destek Sistemidir. Sistem, kullanıcıya hazır bir pilot bölge sunar. Kullanıcı, kod yazmadan, doğrudan harita üzerinden şerit kapatma, yön değiştirme veya yeni bir şerit ekleme gibi geometrik müdahaleler yapar. Sistem bu müdahaleleri ana haritayı bozmadan "izole bir senaryo kopyası" üzerinde uygular. Ağın bağlantısallığı (routing) doğrulandıktan sonra, asenkron iş kuyruğu (job queue) üzerinden simülasyon çalıştırılır. Sonuçlar; ısıl haritalar, gecikme metrikleri ve karşılaştırmalı tabloları ile kullanıcıya sunulur.

3.2 Fonksiyonel Gereksinimler

Id	Gereksinim	Açıklama	Öncelik	Kabul Kriteri
R1	Sabit Ağ Topolojisi Yüklenmesi	Sistem, önceden belirlenmiş pilot bölgenin yol ağını OSM[4] verisinden .net.xml formatına dönüştürebilmeli ve bunu arayüze varsayılan çalışma alanı olarak yükleyebilmelidir.	Yüksek	Kullanıcı arayüzü açtığında seçili pilot bölgenin ağ topolojisi eksiksiz bir harita görünümü olarak render edilmelidir.
R2	Talep Üretimi	Sistem, trafik sayım verisi ve arazi kullanım yoğunluğunu (Gravity Model[5]) kullanarak bir OD matrisi üretmeli, bu matrisi od2trips ile trip tanımlarına, ardından duarouter ile .rou.xml rota dosyasına dönüştürebilmelidir.	Yüksek	Üretilen OD matrisi od2trips aracı kullanılarak geçerli trip XML tanımlarına; ardından duarouter ile hata fırlatmadan .rou.xml dosyasına dönüştürülebilmelidir.

R3	Kontrollü Müdahale Uygulama	Sistem, harita üzerinden yol segmenti/kavşak seçilerek belirtilen katalogdan en az 4 farklı müdahalenin çalışma kopyası üzerinde uygulanmasını sağlamalıdır.	Yüksek	Seçilen müdahale tipi, backend tarafında bir JSON objesi olarak kaydedilmeli ve XML ayrıştırıcısı ile izole .net.xml kopyasına orijinal yapıyı bozmadan (netconvert hataları üretmeden) işlenebilmelidir.
R4	Ağ Doğrulama	Sistem, simülasyon öncesinde çalışma kopyasını bağlantısız yollar veya izole düğümler gibi topolojik hatalara karşı kontrol etmelidir.	Yüksek	Ağ üzerinde araçların rotalama yapmasını engelleyen (izole düğümler, çıkmaz sokaklar vb.) yapısal bir sorun tespit edilirse süreç durdurulmalı ve arayüze HTTP 400 durum kodu ile "Geçersiz Harita Topolojisi" hatası dönülmelidir.
R5	Temel Ağ Bütünlüğü	Sistem, hiçbir işlem sonrasında orijinal temel ağ dosyasını değiştirmemelidir.	Yüksek	Her senaryo komutunda sistem yeni bir "Modified .net.xml" dosyası oluşturmalıdır.
R6	Asenkron Simülasyon Yürütümü	Simülasyon işi bir job kuyruğuna eklenmeli ve backend kullanıcıyı bloke etmemelidir.	Yüksek	API, "Run Simulation" isteğine anında "HTTP 202 Accepted" ve bir Job ID ile yanıt vermelidir. Sistem, bu süreçte arayüzde bir yükleme durumu göstermelidir.

R7	Simülasyon Zaman Aralığı Seçimi	Kullanıcı analiz edilecek zaman aralığını seçebilmelidir.	Orta	Arayüzde yer alan bir zaman seçici bileşeni ile girilen başlangıç ve bitiş saniyeleri, arka planda SUMO'nun --begin ve --end parametrelerine doğru bir şekilde aktarılmalıdır. Sistemde varsayılan olarak Peak Hour (07:00–09:00) seçili gelir.
R8	Metrik Analizi	Sistem, üretilen log dosyalarını otomatik olarak ayrıştırarak yol segmenti seviyesinde trafik yoğunluğu, ortalama hız ve araç hacmini; kavşak seviyesinde ise ortalama gecikme süresi ve kuyruk uzunluğunu hesaplamalıdır.	Yüksek	Simülasyon tamamlandığında sistem, edgedata-output.xml dosyasını ayrıştırarak belirtilen tüm metrikleri (yoğunluk, hız, hacim, gecikme ve kuyruk uzunluğu) eksiksiz olarak hesaplamalı ve sonuçları standart bir JSON formatında arayüze sunmalıdır.
R9	Karşılaştırmalı Görselleştirme	Temel senaryo ile müdahale senaryosunun sonuçları; harita tabanlı ısı haritası, grafik ve karşılaştırma tablosu olarak sunulmalıdır.	Yüksek	Arayüz; tek bir harita görünümü üzerinden, bir buton yardımıyla mevcut durum ile müdahale edilmiş durum arasında geçiş yapılabilmesini sağlamalı ve trafik yoğunluklarını renk skalasıyla gösterebilmelidir.

R10	Kural Tabanlı Uyarı	Gecikme süresi tanımlı bir eşiği aştığında veya yoğunluk %30 arttığında görsel uyarı verilmelidir.	Orta	Sistem, müdahale sonrası hesaplanan metrikleri baz senaryo ile karşılaştırdığında şu iki koşuldan en az biri gerçekleşirse ilgili kavşak veya yol segmenti üzerine tıklanabilir kırmızı bir "Darboğaz Uyarısı" pini eklemelidir: 1) Kavşak gecikme süresinin temel senaryoya göre 60 saniyeden fazla artması. 2) Yol segmenti trafik yoğunluğunun temel senaryoya göre %30'dan fazla artması.

Kaynak: Proje Spesifikasyon Raporu

3.3 Fonksiyonel Olmayan Gereksinimler

Id	Gereksinim	Açıklama
R11	Hacim Tutarlılığı – GEH	Simülasyon çıktıları, gerçek sensör verileriyle GEH istatistiği[6] üzerinden karşılaştırıldığında referans noktalarının en az %65'inde GEH < 10.0 sağlanmalıdır.
R12	OD Dağılım Doğruluğu – KL	Üretilen OD matrisinin uzamsal/zamansal dağılımı, gerçek taksi kayıtlarıyla[7] KL Iraksama metriği[8] üzerinden en az %80 istatistiksel benzerlik sağlamalıdır.

R13	Performans – Simülasyon Süresi	Geliştirme makinesinde (en az 4 Core, 8GB RAM), 40–50 yol segmentlik bir senaryo koşusu, çökmeden 180 saniye içinde tamamlanmalıdır.
R14	Performans – Arayüz Render	Simülasyon tamamlandıktan sonra sonuç görselleştirmeleri (heatmap vb.) kullanıcıya 3 saniye içerisinde sunulmalıdır.
R15	Hata Mesajı Yanıt Süresi	Ağ doğrulama kontrollerinden biri başarısız olduğunda sistem 2 saniye içinde spesifik bir hata mesajı göstermelidir.
R16	Taşınabilirlik	Sistem (Frontend, Backend, Worker) Docker container'ları olarak paketlenmeli ve işletim sistemi bağımsız çalıştırılabilmelidir.
R17	Güvenlik – Girdi Doğrulama ve İzolasyon	Sistem, kullanıcı arayüzünden gelen müdahale verilerini (JSON) arka plandaki netconvert, duarouter ve sumo alt süreç (subprocess) çağrılarına iletmeden önce katı bir şekilde doğrulamalıdır (sanitize). İlgili uç noktalarda (endpoint) komut enjeksiyonu (command injection) ve rastgele dosya sistemi erişimi riskleri %100 engellenmelidir.
R18	Güvenilirlik – Müdahale Başarı Oranı	Spesifikasyonda taahhüt edildiği üzere; sistemde tanımlı olan 6 müdahale tipinden en az 4 farklı tip, izole düğüm hatası üretmedikleri (geçerli bir topoloji oluşturdukları) sürece çalışma kopyası üzerinde %100 başarı oranıyla (SUMO motoru çökmeden veya veri kaybı yaşanmadan) sisteme uygulanabilmeli ve simüle edilebilmelidir.

Kaynak: Proje Spesifikasyon Raporu

3.4 Pseudo Gereksinimler (Teknoloji/Platform Kısıtları)

Sistemin tasarım, mimari ve gerçekleştirme aşamalarında uyulması zorunlu olan operasyonel kısıtlar, platform tercihleri ve teknolojik sınırlamalar aşağıda detaylandırılmıştır. Bu gereksinimler, projenin erişilebilirlik, taşınabilirlik ve düşük maliyet hedeflerini güvence altına almak amacıyla belirlenmiştir.

Lisans ve Maliyet Kısıtı (Tamamen Açık Kaynak Zorunluluğu):

- ➔ **Kısıt:** Projenin hiçbir aşamasında ücretli, ticari veya abonelik gerektiren bir yazılım, kütüphane, harita servisi ya da veritabanı kullanılmayacaktır.
- ➔ **Gerekçe:** Sistemin bütçe kısıtları olan yerel yönetimler ve bağımsız araştırmacılar tarafından ücretsiz olarak kullanılabilmesi hedeflenmektedir. Bu doğrultuda simülasyon motoru olarak Eclipse Public License lisansına sahip SUMO kullanılacaktır. Harita verileri (OpenStreetMap), talep modelleri ve kod tabanında kullanılacak tüm bağımlılıklar açık kaynaklı araçlardan seçilecektir.

Mimari ve Dil Kısıtı (Python ve Asenkron Backend Odaklılık):

- ➔ **Kısıt:** Sistemin sunucu tarafı, Python programlama dili ve FastAPI framework[9] kullanılarak geliştirilecektir.
- ➔ **Gerekçe:** Sistemin temel işlevleri olan veri manipülasyonu, O-D (Origin-Destination) matrisi üretimi ve Gravity Model hesaplamaları için Python ekosistemindeki veri bilimi kütüphaneleri endüstri standartlarıdır. Ayrıca, SUMO motorunun alt süreç olarak asenkron koşturulması ve sonuç loglarının ayrıştırılması Python ile tam uyumludur. Uzun süren simülasyon işlemlerinin web sunucusunu bloklamasını engellemek adına, asenkron yapıyı yerleşik olarak ve yüksek performansla destekleyen FastAPI tercih edilecektir.

Veri Yönetimi ve Depolama Kısıtı (RDBMS Bağımsızlığı):

- ➔ **Kısıt:**Proje kapsamında PostgreSQL, MySQL veya Oracle gibi kompleks, yüksek kaynak tüketen ve bakım maliyeti gerektiren bir İlişkisel Veritabanı Yönetim Sistemi (RDBMS) kullanılmayacaktır.
- ➔ **Gerekçe:** Sistem mimarisinin sade tutulması, bakım maliyetlerinin düşürülmesi ve taşınabilirliğin artırılması hedeflenmektedir. Bu doğrultuda, yol ağları ve rota verileri SUMO'nun doğal formatı olan XML tabanlı dosyalarda tutulacaktır. Kullanıcıların senaryo tercihleri, konfigürasyon dosyaları, asenkron işlerin durum takibi ve simülasyon çıktıları gibi verilerin yönetimi ise tamamen JSON ve log dosyaları üzerinden sağlanacaktır.

Kullanıcı Arayüzü ve Görselleştirme Teknolojileri Kısıtı:

- ➔ **Kısıt:** Web arayüzü React kütüphanesi[10] kullanılarak geliştirilecek; harita tabanlı yoğun render işlemleri için Leaflet.js veya WebGL tabanlı vektör harita motorları zorunlu tutulacaktır.
- ➔ **Gerekçe:** Kullanıcı arayüzü; "tek harita üzerinden senaryo geçişleri", "açılır paneller (sidebar)" ve "gerçek zamanlı durum bildirimleri" gibi karmaşık ve etkileşimli bileşenlerden oluşmaktadır. React'in bileşen tabanlı mimarisi ve sanal DOM yapısı bu karmaşıklığı yönetmek için gereklidir. Öte yandan, simülasyon sonucunda binlerce yol segmentinin üzerindeki trafik yoğunluğunu renkli ısı haritaları olarak göstermek tarayıcı üzerinde ciddi bir bellek yükü oluşturur. Tarayıcının kilitlenmesini önlemek adına performanslı mekânsal görselleştirme kütüphanelerinin kullanımı kritik bir mimari kısıttır.

4. SİSTEM MODELLERİ

4.1. Senaryolar

Bu bölümde, sistemin hedeflenen işlevselliğini doğrulamak amacıyla kullanıcı ile sistem arasındaki etkileşimleri adım adım tanımlayan senaryolar sunulmuştur. Senaryolar; sistemin normal işleyişini gösteren "Mutlu Yollar (Happy Paths)" ve sistemin hatalı kullanıcı girdilerine karşı kendini nasıl koruduğunu gösteren "Hata Yolları (Error Paths)" olarak ikiye ayrılmıştır.

Senaryo 1: Başarılı Şerit Kapatma ve Simülasyon (Mutlu Yol - Road Diet Analizi)

Aktörler: Ulaşım Mühendisi / Şehir Planlamacısı (Kullanıcı), Karar Destek Sistemi

Ön Koşul: Kullanıcı sisteme giriş yapmış, çalışılacak pilot bölge haritası (Base Scenario) başarıyla yüklenmiş ve mevcut durumun referans metrikleri arka planda hazırdır.

Tetikleyici (Trigger): Kullanıcı, trafik sıkışıklığı olan ana bir arterde yayalaştırma projesi kapsamında bir şeridin iptal edilmesinin etkilerini görmek ister.

Adımlar:

1. Kullanıcı, harita üzerinde (UI) müdahale etmek istediği yol segmentine (edge) tıklar.
2. Ekranın sağ tarafında açılan "Müdahale Kataloğu" panelinden **"Sağ Şeridi Kapat (Road Diet)"** eylemini seçer. Seçilen müdahale, "Aktif Senaryo" listesine eklenir.
3. Kullanıcı, ekranın sağ üst köşesinde bulunan "Analiz Et / Simüle Et" butonuna tıklar.
4. Sistem (Backend), bu müdahaleyi orijinal topolojiyi bozmadan bağımsız bir Modified .net.xml (çalışma kopyası) dosyasına yazar.
5. Ağ doğrulama (Network Validation) modülü çalışır ve bağlantı kopukluğu olmadığını onaylar.
6. İstek, asenkron İş Kuyruğuna (Job Queue) iletilir. Kullanıcı arayüzünde "Simülasyon Sürüyor, Rotalar Hesaplanıyor..." şeklinde bir yükleme ekranı belirir.
7. Ortalama 2-3 dakika içinde SUMO motoru çalışmasını tamamlar, loglar ayrıştırılır ve arayüze veri gönderilir.

Son Koşul : Sistem ısı haritasını (heatmap) günceller. Kullanıcı, kapattığı şeridin doğrudan üzerindeki araç sayısının sıfırlandığını, ancak paralel caddelerdeki (alternatif rotalardaki) trafik yoğunluğunun ve bekleme sürelerinin arttığını görsel renk skalası üzerinden tespit eder.

Senaryo 2: Katılım Şeridi (Slip-Lane) Eklenmesi (Mutlu Yol - Geometrik İyileştirme)

Aktörler: Ulaşım Mühendisi, Karar Destek Sistemi

Ön Koşul: Pilot bölge yüklü durumdadır ve kullanıcı temel senaryonun trafik darboğazlarını (kırmızı düğümler) incelemektedir.

Tetikleyici: Kullanıcı, sağa dönüşlerde yaşanan yığılmayı çözmek için spesifik bir kavşağa geometrik bir müdahale (slip-lane) yapmak ister.

Adımlar:

1. Kullanıcı, aşırı sıkışık (kırmızı) olarak işaretlenmiş bir kavşak noktasına (node) tıklar.
2. Açılan özellikler panelinden "U-Dönüş Cebi / Katılım Şeridi (Slip-Lane) Ekle" hazır şablonunu seçer.
3. Kullanıcı simülasyonu başlatır. Sistem, düğüm noktasının XML tanımını günceller ve iteratif DUA (Dinamik Kullanıcı Dengesi)[11] rotalama algoritmasını arka planda çalıştırır.
4. İşlem başarıyla tamamlandığında, sonuç ekranında bir geçiş butonu (toggle) belirir; kullanıcı bu buton ile aynı harita üzerinde "Temel Senaryo" ve "Müdahale Senaryosu" görünümleri arasında geçiş yapabilir.

Son Koşul: Kullanıcı, tek harita üzerinde toggle butonuna basarak önce mevcut durumu, ardından müdahale edilmiş durumu inceler. Kullanıcı alt paneldeki metrik tablosunda; "Baz Senaryoda Ortalama Gecikme: 45 sn" değerinin, "Müdahale Senaryosunda Ortalama Gecikme: 28 sn" değerine düştüğünü matematiksel olarak doğrular ve bu altyapı yatırımının faydalı olduğuna karar vererek raporu dışa aktarır.

Senaryo 3: Kural Tabanlı Uyarının Tetiklenmesi (Mutlu Yol - Otonom Analiz)

Aktörler: Şehir Planlamacısı, Sistemin Analiz ve Uyarı Modülü

Ön Koşul: Kullanıcı, ağ üzerinde topolojik bir değişiklik yapmış ve simülasyon sonuç ekranına (heatmap) ulaşmıştır.

Tetikleyici: Kullanıcının yaptığı bir müdahale (örneğin bir ana yolun tek yöne çevrilmesi), öngörülemeyen ikincil bir etki yaratarak başka bir bölgede ani bir trafik yığılmasına sebep olmuştur.

Adımlar:

1. Kullanıcı ana yollardan birini "Tek Yön" olarak düzenler ve simülasyonu başlatır.
2. Simülasyon tamamlandıktan sonra Backend modülü, Baz Senaryo logları ile yeni senaryo loglarını düğüm bazlı (node-by-node) kıyaslar.
3. Sistem, kullanıcının müdahale ettiği noktadan 2 kilometre kuzeydeki bir kavşakta, yoğunluğun ve bekleme süresinin baz senaryoya göre %35 arttığını tespit eder.
4. Arayüz render edilirken, bu spesifik kuzey kavşağının üzerine animasyonlu (yanıp sönen) ve kırmızı bir **Uyarı Pini (!)** yerleştirilir.

Son Koşul: Kullanıcı, normalde gözden kaçırabileceği bu uzak etkiyi fark edip pine tıklar. Sistem şu mesajı gösterir: *"Darboğaz Uyarısı: Trafik yoğunluğu %30 barajını aştı. Ortalama Gecikme: +120 saniye."* Kullanıcı bu bilgiyle tasarımını revize etme kararı alır.

Senaryo 4: İzole Düğüm Hatası ve Geçersiz Topoloji (Hata Yolu - Güvenlik Ağı)

Aktörler: Kullanıcı, Ağ Doğrulama Servisi

Ön Koşul: Kullanıcı, pilot bölge haritası üzerinde aktif olarak çalışmaktadır.

Tetikleyici: Kullanıcı, yanlışlıkla veya deneme amacıyla, belirli bir mahalleye araç girişini sağlayan **tek** yolu trafiğe kapatır.

Adımlar:

1. Kullanıcı, X mahallesine giden ana giriş yolunu seçer ve "Trafiğe Kapat" komutunu verir.
2. Sağ üst köşedeki "Simüle Et" butonuna basar.
3. Backend istek aldığında, hemen Job Queue'ya göndermek yerine netconvert aracı ile Ağ Doğrulama sürecini başlatır.
4. Doğrulama algoritması, X mahallesindeki düğümlerin hiçbir dış bağlantısı kalmadığını (İzole Düğüm / Disconnected Subgraph) ve OD matrisindeki araçların buraya asla rotalama yapamayacağını tespit eder.

Son Koşul: Simülasyon motorunun (SUMO) çökelmesini ve sistem kaynaklarının boşa harcanmasını önlemek adına işlem anında iptal edilir. Kullanıcının ekranında 2 saniye içerisinde bir Hata Modalı (Modal) açılır ve şu net eylem çağrısı sunulur: *"Simülasyon Başlatılamadı: Geçersiz Harita Topolojisi. Yaptığınız müdahale, işaretli mahalleye araç erişimini tamamen kesmektedir (İzole Düğüm). Lütfen analize devam etmek için bölgeye alternatif bir giriş yolu sağlayın veya mevcut müdahaleyi geri alın."*

4.2 Use Case Model

Kullanım durumu modeli, sistemin fonksiyonel gereksinimlerini son kullanıcı ve dış sistemler perspektifinden tanımlar. Bu bölüm, sistemin sınırlarını çizerken; kimin hangi işlemi yapmaya yetkili olduğunu, eylemlerin ön koşullarını ve işlemler arası bağımlılıkları (include/extend) standartlaştırmaktadır.

4.2.1. Aktörler

Sistemdeki etkileşimler, doğrudan eylemi başlatan insan aktörler ile arka planda hesaplama yapan sistem aktörleri arasında gerçekleşmektedir.

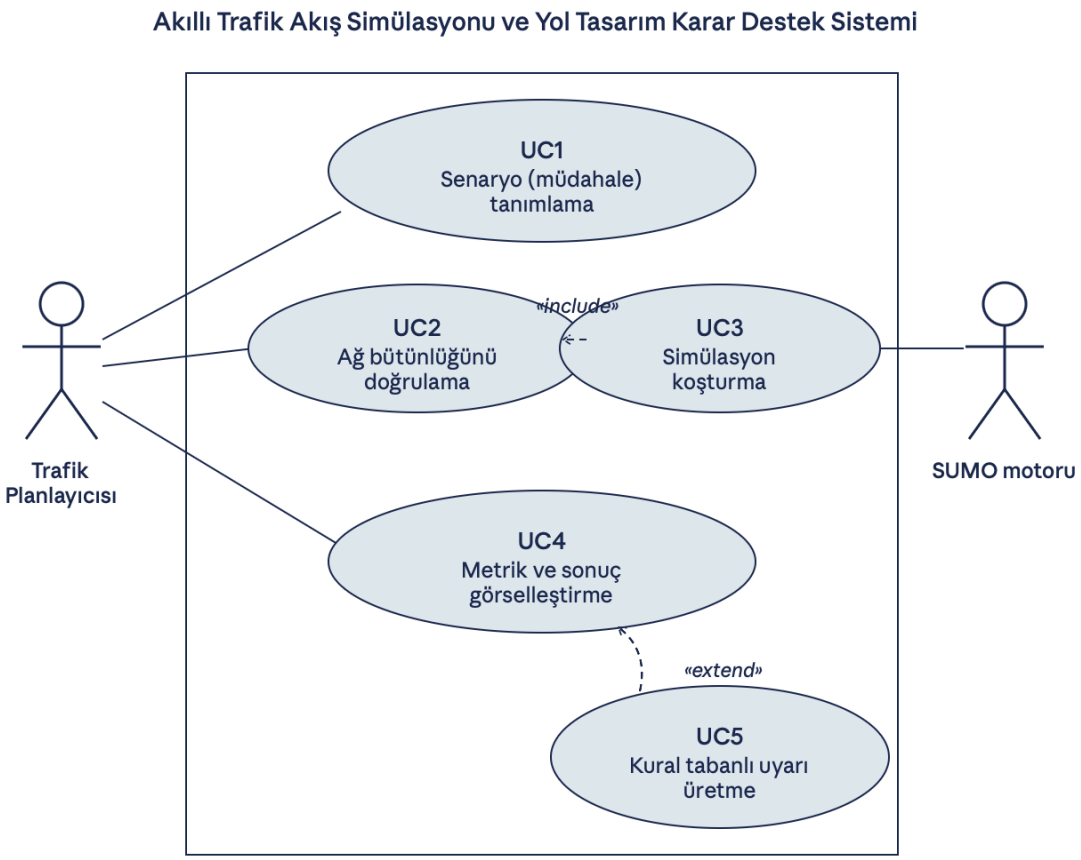
Trafik Planlayıcısı (Birincil/İnsan Aktör): Sistemi grafik arayüz (UI) üzerinden doğrudan kullanan hedef kullanıcıdır (şehir planlamacıları, ulaştırma mühendisleri vb.). Sisteme iş kurallarını (müdahaleleri) girer, analiz edilecek bölgeyi/zamanı seçer, asenkron simülasyon sürecini başlatır ve sistemin ürettiği makroskobik çıktıları karar verme süreçlerinde kullanır.

SUMO Motoru (İkincil/Sistem Aktörü): Sistemin çekirdek simülasyon yeteneğini sağlayan dış/arka plan aktörüdür. Arayüzden bağımsız çalışan bir "kara kutu" olarak modellenmiştir. Trafik Planlayıcısı tarafından tetiklenen işleri Job Queue üzerinden alır, mikroskobik araç fiziği ve Dinamik Kullanıcı Dengesi (DUA) iterasyonlarını hesaplar ve sonuçları ham XML logları olarak üreterek sisteme geri döndürür.

4.2.2 Use Case Diyagramı

Aşağıdaki diyagram, sistem sınırını (system boundary), iki aktörü ve altı kullanım durumunu; aralarındaki «include» (zorunlu içerme) ve «extend» (koşullu genişletme) ilişkileriyle birlikte göstermektedir. UC3, başlamadan önce UC2'yi zorunlu olarak içerir; UC6 (Kural Tabanlı Uyarı Üretme) ise UC4'ün sonucuna bağlı olarak koşullu şekilde tetiklendiğinden «extend» ilişkisiyle bağlanmıştır.

4.2.3. Kullanım Durumları



Şekil 2. Kullanım durumu (use case) diyagramı

Sistemin temel işlevleri, birbirleriyle olan mantıksal ilişkileri ve akışları göz önünde bulundurularak aşağıdaki gibi detaylandırılmıştır:

UC1: Senaryo (Müdahale) Tanımlama

İlgili Aktörler: Trafik Planlayıcısı

Kısa Açıklama: Kullanıcının, mevcut pilot harita üzerinde trafik sıkışıklığını çözmek amacıyla fiziksel/geometrik değişiklikler (yol kapatma, şerit ekleme vb.) tasarlaması sürecidir.

Ön Koşul: Sisteme giriş yapılmış ve analiz edilecek Temel Senaryo (Base Network) haritaya hatasız şekilde yüklenmiş olmalıdır.

Ana Akış :

1. Kullanıcı harita üzerinden müdahale edilecek Edge (yol) veya Node (yol bağlantısı) birimini seçer.
2. Sistem, seçilen birime uygun müdahale opsiyonlarını (Katalog) listeler.
3. Kullanıcı katalogdan eylemi seçer ve "Listeye Ekle" komutunu verir.

Alternatif Akış: Kullanıcı harita sınırları dışındaki bir birimi seçmeye çalışırsa, sistem eylemi reddeder ve "Geçersiz bölge" uyarısı verir.

Son Koşul (Post-condition): Seçilen müdahaleler JSON objesi formatında geçici belleğe (State/Context) alınır, orijinal topoloji henüz değiştirilmez.

UC2: Ağ Bütünlüğünü Doğrulama

İlgili Aktörler: İlgili Aktörler: Yok (Dahili Süreç)

Kısa Açıklama: Kullanıcının tanımladığı (UC1) müdahalelerin, fiziksel olarak geçerli bir ağ yapısı oluşturup oluşturmadığının kontrol edilmesidir. Sistem içi otomatik bir güvenlik sürecidir.

Ön Koşul: Kullanıcının "Simülasyonu Başlat" butonuna tıklamış olması gereklidir.

Ana Akış (Main Flow):

1. Sistem, Base.net.xml dosyasının bir çalışma kopyasını çıkarır.
2. Müdahaleler bu kopyaya işlenir.
3. SUMO netconvert aracı kullanılarak ağdaki bağlantısal bütünlük (izole düğüm veya ulaşılamaz bölgeler) taranır.

Alternatif Akış: Eğer çalışma kopyasında izole düğüm bulunursa sistem HTTP 400 hatası döndürür ve simülasyon onayını iptal eder.

Son Koşul: Ağ geçerli ise süreç UC3'e geçiş onayı verir. Geçersiz ise işlem durdurulur ve kullanıcıya HTTP 400 hatası ile spesifik bir uyarı mesajı döndürülür.

UC3: Simülasyon Koşturma

İlgili Aktörler: Trafik Planlayıcısı (Tetikleyici), SUMO Motoru (İşleyici)

İlişkiler: Bu kullanım durumu, başlamak için zorunlu olarak <<include>> UC2 (Ağ Bütünlüğünü Doğrulama) adımını içerir. Doğrulama olmadan simülasyon başlatılamaz.

Kısa Açıklama: Doğrulanan topolojinin ve önceden üretilen O-D talep matrisinin, simülasyon motoruna asenkron bir iş (background job) olarak gönderilmesi ve işlenmesi sürecidir.

Ön Koşul: Ağın UC2'den onay almış olması ve sentetik taleplerin (.rou.xml) hazır olması gerekir.

Ana Akış (Main Flow):

1. Sistem işi Message Broker (iş kuyruğu) üzerine yazar.
2. Kullanıcı arayüzüne "202 Accepted" dönülerek ekran bloke edilmeden bir bekleme (loading) statüsüne geçilir.
3. SUMO Motoru kuyruktaki işi alır ve iteratif hesaplamaları tamamlar.

Alternatif Akış: SUMO motoru 180 saniye içerisinde (Zaman Aşımı) yanıt vermezse, Worker işi keser (Abort) ve arayüze "Simülasyon Başarısız" mesajı iletilir.

Son Koşul: İşlem hatasız bittiğinde SUMO, ham istatistikleri içeren fcd-output.xml ve edgedata-output.xml dosyalarını diske yazar.

UC4: Metrik ve Sonuç Görselleştirme

İlgili Aktörler: Trafik Planlayıcısı

Kısa Açıklama: SUMO motorunun ürettiği okunması zor ham log dosyalarının, karar destek mekanizmasına uygun görsel ve sayısal analizlere dönüştürülmesi işlemidir.

Ön Koşul: UC3 (Simülasyon Koşturma) işleminin başarıyla "Completed" statüsüne ulaşmış ve çıktıları üretmiş olması gereklidir.

Ana Akış (Main Flow):

1. Backend ayrıştırıcı (parser) modülü XML dosyalarını okur, düğüm ve kenar bazlı yoğunluk/gecikme metriklerini toplar (aggregation).
2. Elde edilen veriler standardize edilmiş bir JSON yanıtı olarak Frontend'e iletilir.
3. React tabanlı arayüz, bu verileri mevcut harita üzerinde ısı haritalarına dönüştürerek render eder.

Alternatif Akış: XML log dosyaları bozuk veya eksik üretilmişse, Parser hatayı yakalar ve UI'a "Metrikler Yüklenemedi" uyarısı düşer.

Son Koşul: Kullanıcıya; tek bir harita üzerinde bir geçiş butonu ile Base ve Modified senaryolar arasında geçiş yapılabilen karşılaştırmalı görünüm, metrik tabloları ve sistem tarafından otomatik hesaplanmış darboğaz uyarıları sunulur.

UC5: Kural Tabanlı Uyarı Üretme

İlgili Aktörler: Sistem (Analiz ve Uyarı Modülü)

İlişkiler: Bu kullanım durumu, UC4'ün tamamlanmasının ardından koşullu olarak tetiklendiğinden UC4'ü «extend» eder; her simülasyonda değil, yalnızca eşik aşımı gerçekleştiğinde devreye girer.

Kısa Açıklama: Simülasyon sonucu üretilen metriklerin, önceden tanımlanmış eşik değerlerine (yoğunluk artışı, gecikme süresi) karşı otomatik olarak taranmasıdır.

Ön Koşul: UC4 tamamlanmış ve Base ile Modified senaryoların metrikleri karşılaştırmaya hazır olmalıdır.

Ana Akış:

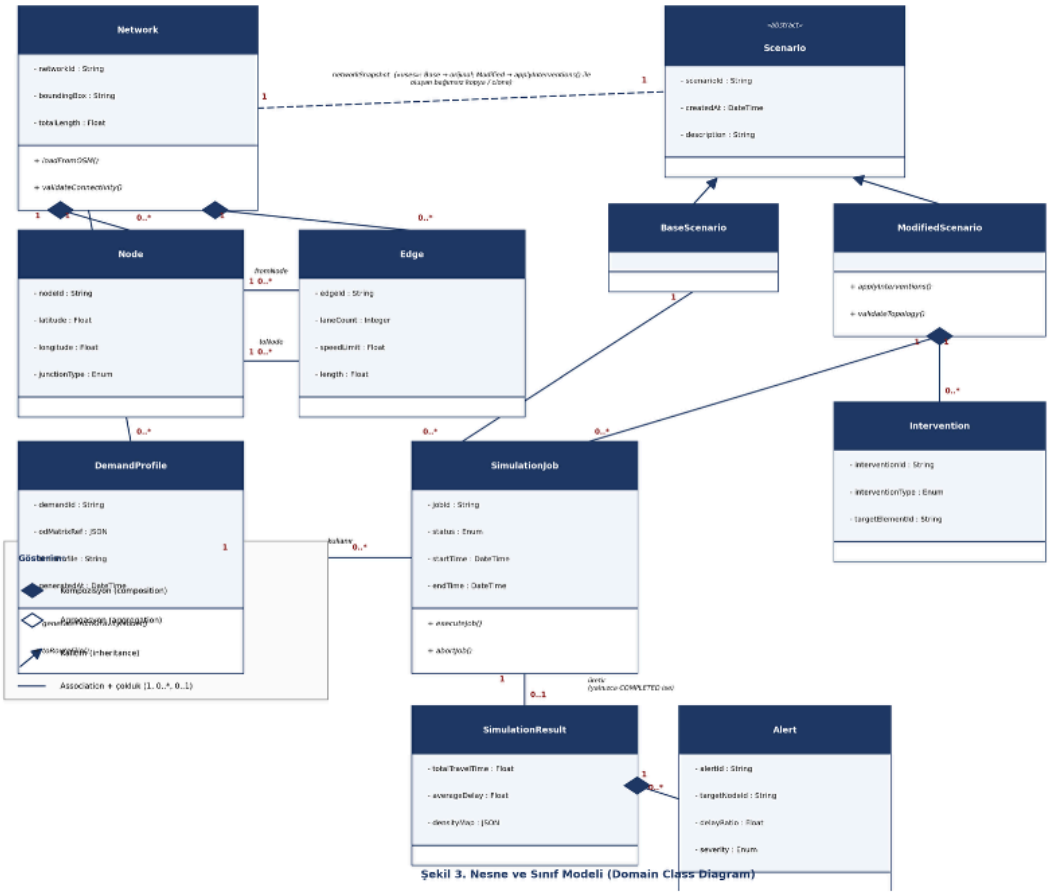
1. Sistem, Modified senaryodaki her düğüm/kenar için Base senaryoya göre yüzdesel değişimi hesaplar.
2. Yoğunluk artışı %30'u veya kavşak gecikmesi tanımlı eşiği aştığında, ilgili konum "darboğaz" olarak işaretlenir.
3. Arayüzde, işaretlenen konumların üzerine animasyonlu kırmızı uyarı pinleri (marker)yerleştirilir.

Alternatif Akış: Hiçbir kavşakta eşik değeri aşılmazsa, bu kullanım durumu tetiklenmez.

Son Koşul: Kullanıcı, kırmızı pine tıkladığında darboğazın büyüklüğünü ve olası nedenini açıklayan bir bilgi balonu (tooltip) görüntüler.

4.3. Nesne ve Sınıf Modeli (Domain Modeli)

Bu bölümdeki sınıf diyagramı (Class Diagram), sistemin kalıcılık (veritabanı tabloları) veya sunum (arayüz) katmanlarından bağımsız olarak, iş mantığını (business logic) işletirken bellekte tuttuğu temel kavramları (entities) modeller. Domain-Driven Design (DDD) prensiplerine göre hazırlanan bu model; sistemin kavramsal sözlüğünü oluşturur, nesnelerin sahip olduğu yapısal özellikleri (nitelikler), davranışları (metotlar) ve birbirleriyle olan yapısal ilişkileri (kalıtım, kompozisyon, agregasyon) tanımlar.



Şekil 3. Nesne ve Sınıf Modeli (Domain Class Diagram)

4.3.2. Sınıf Tanımları ve Sorumluluklar

Network (Ağ)

Belirli bir pilot bölgedeki tüm yol sistemini ve topolojiyi kapsayan en üst düzey (şemsiye) nesnedir.

- **Nitelikler:** networkId (String), boundingBox (String), totalLength (Float).
- **Metotlar:** loadFromOSM(), validateConnectivity().
- **İlişkiler:** Bir Network, kompozisyon ilişkisiyle (1 → 0..*) çok sayıda Node ve Edge nesnesi barındırır; Network silindiğinde bağlı Node/Edge nesneleri de anlamını yitirir. Ayrıca her

Scenario (1 → 1) tam olarak bir Network anlık görüntüsüne (networkSnapshot) referans verir — bkz. Scenario tanımındaki kopyalama (clone) semantiği.

Node (Düğüm)

Sistemdeki kavşak noktalarını, çıkmaz sokak uçlarını veya yol birleşim yerlerini ifade eder.

- **Nitelikler:** nodeId (String), latitude (Float), longitude (Float), junctionType (Enum: TRAFFIC_LIGHT, PRIORITY, ROUNDABOUT).
- **İlişkiler:** Bir Node, 0..* Edge'in fromNode ucu ve ayrıca 0..* Edge'in toNode ucu olabilir. fromNode ve toNode, anlamca farklı iki rol olduğundan diyagramda iki ayrı, rol adlı association olarak gösterilmiştir (bkz. Edge tanımı).

Edge (Yol Segmenti / Kenar)

İki düğümü (Node) birbirine bağlayan yönlü araç yollarını temsil eder. Sürüş fiziğinin gerçekleştiği temel birimdir.

- **Nitelikler:** edgeId (String), laneCount (Integer), speedLimit (Float — km/s), length (Float — metre).
- **İlişkiler:** Her Edge, biri fromNode diğeri toNode rolünde olmak üzere iki ayrı association ile tam olarak 2 farklı Node nesnesine bağlanır: Edge (0..*) — (1) Node [fromNode] ve Edge (0..*) — (1) Node [toNode]. Bu ayrım, başlangıç ve bitiş uçlarının diyagram düzeyinde karıştırılmasını önler.

DemandProfile (Talep Profili)

R2'de tanımlanan talep üretimi sürecinin çıktısını temsil eden domain sınıfıdır; Gravity Model ile üretilen OD matrisini ve bu matristen türetilen zaman profilini kapsar.

- **Nitelikler:** demandId (String), odMatrixRef (JSON — çekim modeli çıktısı), timeProfile (String — örn. "Peak Hour 07:00–09:00"), generatedAt (DateTime).
- **Metotlar:** generateFromGravityModel() (Trafik hacmi ve arazi kullanım verisinden OD matrisini üretir), toRouteFile() (od2trips/duarouter zincirini tetikleyerek .rou.xml dosyasına dönüştürür).
- **İlişkiler:** Bir Network (1), zaman içinde 0..* DemandProfile üretebilir; bir DemandProfile (1), 0..* SimulationJob tarafından kullanılabilir (aynı talep profili farklı senaryolarda tekrar kullanılabilir).

Scenario (Senaryo — Soyut Sınıf / Abstract Class)

Kullanıcının çalışma alanını ve simülasyonun üzerine kurulacağı topolojik temeli ifade eder. Doğrudan örneği (instance) oluşturulmaz, kalıtım yoluyla alt sınıflara ayrılır.

- **Nitelikler:** scenarioId (String), createdAt (DateTime), description (String).
- **İlişkiler (Network kopyalama semantiği):** Scenario (1) — Network (1) association'ı, R5'in ("orijinal ağ asla değiştirilemez") ve UC1'in ("orijinal topoloji henüz değiştirilmez") gerektirdiği kopyalama davranışını netleştirmek için soyut sınıf düzeyinde tanımlanmıştır: BaseScenario'nun bağlı olduğu Network, OSM'den çekilen değiştirilemez orijinal nesnedir; ModifiedScenario ise applyInterventions() ilk çağrıldığında bu orijinalden türetilen bağımsız

bir Network kopyası (clone) oluşturur ve kendi networkSnapshot'ına bağlanır. Böylece iki alt sınıf hiçbir zaman aynı Network nesnesini paylaşmaz.

BaseScenario (Temel Senaryo)

Scenario sınıfından kalıtım alır. Hiçbir müdahale içermeyen, haritanın OSM'den çekilmiş ham ve değiştirilemez (immutable) orijinal durumunu temsil eder; bağlı olduğu Network nesnesi salt-okunurdur ve hiçbir Intervention tarafından değiştirilemez.

ModifiedScenario (Müdahale Senaryosu)

Scenario sınıfından kalıtım alır. BaseScenario'nun bir kopyası (clone) olarak başlar ve kullanıcının yaptığı değişiklikleri barındırır.

- **Metotlar:** applyInterventions() (Değişiklikleri, BaseScenario'nun Network'ünden türetilen bağımsız kopya üzerine işler), validateTopology() (Yeni ağda izole düğüm olup olmadığını kontrol eder).
- **İlişkiler:** İçerisinde kompozisyon ilişkisiyle (1 → 0..*) birden fazla Intervention nesnesi barındırır. Aggregation değil composition tercih edilmiştir, çünkü bir Intervention sahibi olan ModifiedScenario dışında anlamlı/bağımsız bir varlık değildir ve ModifiedScenario silindiğinde ilişkili tüm Intervention'lar da silinmelidir.

Intervention (Müdahale)

Kullanıcının ModifiedScenario üzerinde yaptığı yapısal/geometrik kural değişikliklerini temsil eder.

- **Nitelikler:** interventionId (String), interventionType (Enum), targetElementId (Müdahalenin uygulandığı Edge veya Node ID'si — tip güvenliği için aşağıdaki eşleme tablosuna bakınız).

Tip güvenliğini modelde açıkça izlenebilir kılmak amacıyla, her interventionType değerinin hangi ağ elemanı türünü (Edge veya Node) hedeflediği aşağıda netleştirilmiştir:

interventionType	**Hedef Eleman Tipi**	**Açıklama**
LANE_CLOSURE_OR_ADDITION	Edge	Şerit sayısını (laneCount) artırır/azaltır.
ONE_WAY_CONVERSION	Edge	Yolu tek yönlü hale getirir.
ROAD_TYPE_AND_SPEED_CHANGE	Edge	Hız sınırını (speedLimit) günceller.

JUNCTION_CONVERSION	Node	Kavşak tipini (junctionType) değiştirir.
SLIP_LANE_ADDITION	Node	Kavşağa katılım şeridi ekler.
BYPASS_ADDITION	Node	Kavşağa bağlantı/bypass yolu ekler.

Kaynak: Kendi Çalışmamız

İleri bir geliştirme adımı olarak, Edge ve Node'u ortak biçimde ele alabilecek soyut bir NetworkElement üst tipi tanımlanıp Intervention'ın targetElementId'si bu tipe association ile bağlanarak tip güvenliği derleme/model düzeyine taşınabilir; capstone kapsamında yukarıdaki eşleme tablosu bu ihtiyacı karşılamaktadır.

SimulationJob (Simülasyon İşi)

Asenkron mimaride iş kuyruğu üzerinde koşturulan iş nesnesidir. Bir senaryonun SUMO motorunda işlenme sürecini kapsar.

- **Nitelikler:** jobId (String), status (Enum: PENDING, RUNNING, COMPLETED, FAILED), startTime (DateTime — işin wall-clock başlama anı), endTime (DateTime — işin wall-clock bitiş anı).
- **Metotlar:** executeJob() (SUMO'yu subprocess ile başlatır), abortJob() (Hata veya zaman aşımı durumunda işi keser ve status'ü FAILED yapar).
- **İlişkiler:** Bir BaseScenario veya ModifiedScenario (1) tarafından 0..* SimulationJob tetiklenebilir; bir DemandProfile (1) 0..* SimulationJob tarafından kullanılabilir. Her SimulationJob en fazla 1 SimulationResult üretir (1 → 0..1) — job PENDING, RUNNING veya FAILED durumundayken henüz/hiç bir sonuç nesnesi bulunmaz; yalnızca status = COMPLETED olduğunda tam olarak 1 SimulationResult üretilmiş olur.

SimulationResult (Sonuç ve Metrikler)

Simülasyon tamamlandığında logların ayrıştırılmasıyla oluşturulan ve arayüze aktarılan veri transfer nesnesidir.

- **Nitelikler:** totalTravelTime (Float), averageDelay (Float), densityMap (JSON — düğüm/kenar bazlı yoğunluk verisi).
- **İlişkiler:** Bir SimulationResult (1), kompozisyon ilişkisiyle 0..* Alert barındırır (R10/UC6/Senaryo 3'ün kural tabanlı uyarı çıktısı için — bkz. Alert tanımı).

Alert (Uyarı)

R10, UC6 ve Senaryo 3'te tanımlanan kural tabanlı darboğaz uyarısının veri karşılığıdır; Ekran S5'te gösterilen uyarı pini ve tooltip içeriği bu sınıftan beslenir.

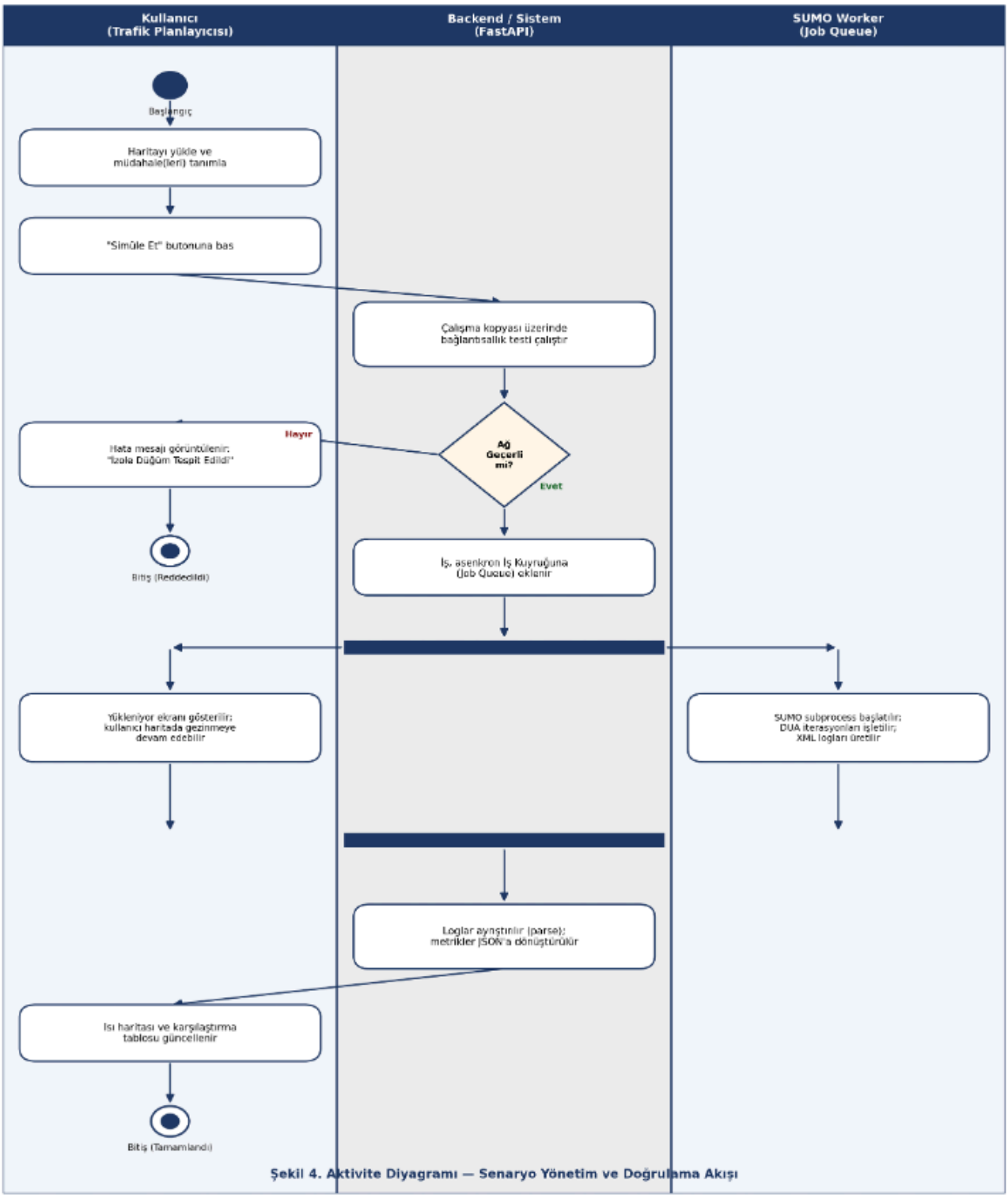
- **Nitelikler:** alertId (String), targetNodeId (String — uyarının tetiklendiği kavşak), delayRatio (Float — Müdahale Sonrası Gecikme / Baz Gecikme oranı), severity (Enum: WARNING, CRITICAL).
- **İlişkiler:** Sahibi olan SimulationResult'tan bağımsız var olamaz (composition); SimulationResult silindiğinde ilişkili tüm Alert'ler de silinir.

4.4 Dinamik Modeller (Dynamic Models)

Sistemin yapısal (statik) tasarımını gösteren Sınıf Modelinin aksine, Dinamik Modeller sistemin zaman içindeki davranışını, durum değişimlerini ve farklı aktörler/bileşenler arasındaki etkileşim akışlarını tanımlar. Bu bölümde sistemin karmaşık iş mantığı; algoritmik akışı gösteren **Aktivite Diyagramı** ve bileşenler arası zaman sıralı mesajlaşmayı gösteren **Ardışık Diyagram** ile soyutlanmıştır.

4.4.1. Aktivite Diyagramı: Senaryo Yönetim ve Doğrulama Akışı

Aktivite diyagramı, kullanıcının bir müdahale tanımlamasından başlayarak olası hata yollarını ve eşzamanlı işlemleri adım adım gösteren iş akışı modelidir. Diyagram üç sorumluluk şeridine (swimlane) ayrılmıştır: Kullanıcı, Backend/Sistem ve SUMO Worker. Özellikle ağ doğrulama aşamasındaki karar düğümü (decision node) ile simülasyonun arka planda çalışırken kullanıcı arayüzünün nasıl tepki verdiğini gösteren çatallanma/birleşme (fork/join) çubukları vurgulanmıştır.



Şekil 4. Aktivite Diyagramı — Senaryo Yönetim ve Doğrulama Akışı

İş Akışı Detayları:

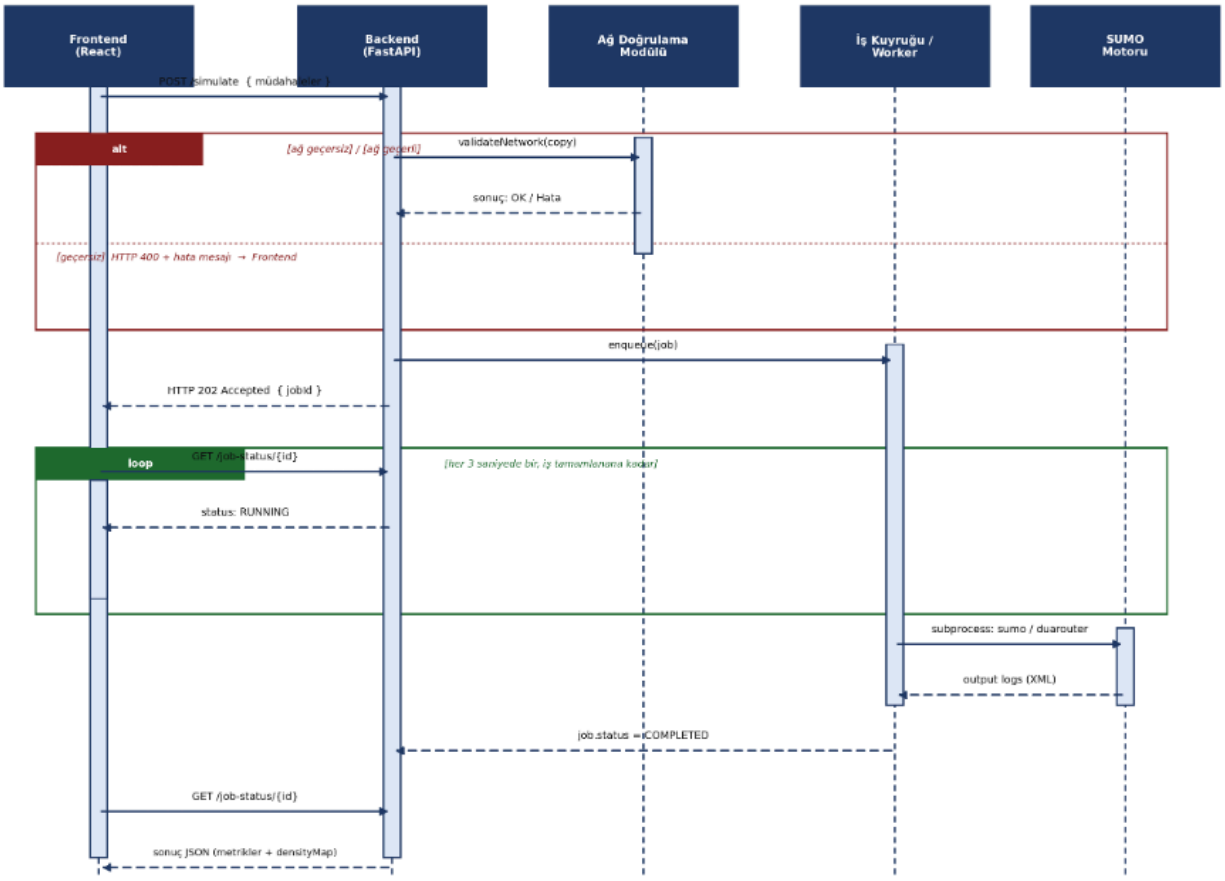
1. **Sürecin Başlaması:** Kullanıcı, arayüz üzerinden haritayı yükler ve bir veya birden fazla topolojik müdahale (şerit kapatma, tek yön yapma) tanımlar.
2. **Karar Düğümü (Decision - Ağ Geçerliliği):** Kullanıcı "Simüle Et" butonuna bastığında, sistem anında simülasyonu başlatmaz. Önce çalışma kopyası üzerinde bir bağlantısallık testi yapar.

- ***Geçersiz Topoloji (Hata Yolu):*** Eğer müdahale bir bölgeye erişimi tamamen kesiyorsa (İzole Düğüm), süreç "Simülasyon" aşamasına geçmeden reddedilir ve kullanıcıya UI üzerinden spesifik bir hata mesajı dönülür.
- ***Geçerli Topoloji (Mutlu Yol):*** Ağ kurallara uygunsa, iş paketlenir ve asenkron kuyruğa (Job Queue) aktarılır.

3. **Paralel İşleme:** İş kuyruğa atıldıktan sonra süreç iki koldan eşzamanlı ilerler:
   - ***Kullanıcı Süreci:*** Arayüz kilitlenmez. Kullanıcıya bir "Yükleniyor (Loading)" ekranı gösterilirken, kullanıcı harita üzerinde başka alanları incelemeye devam edebilir.
   - ***Sistem Süreci:*** Arka plandaki Worker, SUMO motorunu alt süreç (subprocess) olarak başlatır, DUA iterasyonlarını işletir ve XML loglarını üretir.
4. **Birleşme ve Sonlanma:** Arka plan işi bittiğinde, üretilen XML dosyaları ayrıştırılır (parsing), elde edilen metrikler (JSON) arayüze aktarılır ve ısı haritası güncellenerek akış sonlandırılır.

4.4.2. Ardışık Diyagram : Simülasyon İstek Döngüsü

Ardışık diyagram, istemci (Frontend), sunucu (FastAPI), ağ doğrulama modülü, görev yöneticisi (Job Queue/Worker) ve simülasyon motoru (SUMO) arasındaki zaman sıralı (time-ordered) mesajlaşmayı ve API çağrılarını gösterir. Diyagramda iki UML çerçevesi kullanılmıştır: senkron doğrulamayı temsil eden bir alt (alt) çerçevesi ve durum yoklamasını (polling) temsil eden bir döngü (loop) çerçevesi. Bu model, asenkron mimarinin — istemcinin sunucuyu beklerken kilitlenmemesinin — teknik olarak nasıl kurgulandığını açıklar.



Şekil 5. Ardışık (Sequence) Diyagramı — Simülasyon İstek Döngüsü

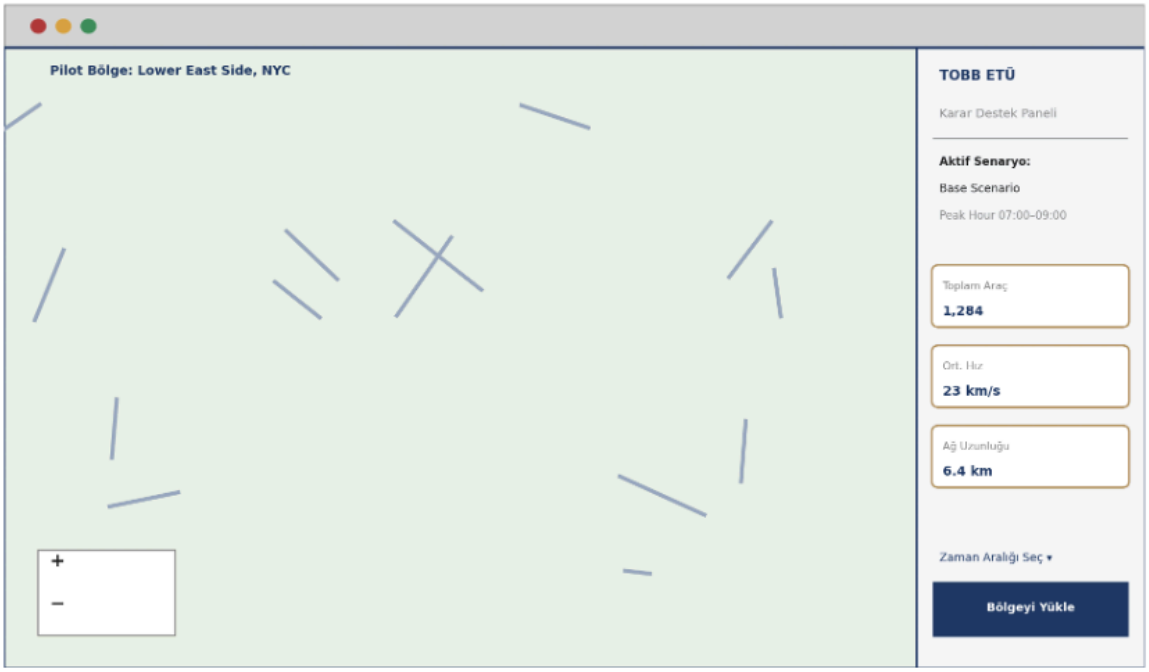
Zaman Sıralı İletişim Detayları:

1. **İstek Gönderimi:** Kullanıcı arayüzü (React), müdahale parametrelerini bir JSON paketi olarak /simulate POST uç noktasına (endpoint) gönderir.
2. **Senkron Doğrulama (Alt/Else Bloku):** Backend ağın geçerli olup olmadığını kendi içinde senkron (bekleyerek) kontrol eder. Hata varsa anında HTTP 400 döner.
3. **Asenkron İş Atama ve Polling (Döngü):** Geçerli bir istek durumunda, Backend işi Worker'a devreder ve UI'a işin kimliğini (Job ID) içeren bir HTTP 202 (Accepted) yanıtı döner.
4. **Durum Yoklaması:** SUMO motoru (kara kutu) saatlerce sürebilecek bir işlem yaparken, UI her 3 saniyede bir /job-status/{id} uç noktasına GET isteği atarak işin durumunu sorar (Polling mekanizması).
5. **Tamamlanma:** Worker işi bitirip veritabanındaki/bellekteki iş durumunu COMPLETED olarak güncellediğinde, UI'ın yaptığı bir sonraki GET isteği nihai analiz sonuçlarını alır ve ekranı çizer.

4.5. Kullanıcı Arayüzü — Navigasyon ve Mock-up Hiyerarşisi

Projenin Kullanıcı Arayüzü (UI), arka planda çalışan karmaşık veri mühendisliği ve simülasyon hesaplamalarını son kullanıcıdan (şehir plancısı) soyutlayarak, tamamen "Görsel Öncelikli (Visual-First)" bir etkileşim modeli sunmak üzere tasarlanmıştır. Kullanıcı, uzun formlar doldurmak veya komut satırı kullanmak yerine; doğrudan harita üzerindeki nesnelerle etkileşime geçerek karar süreçlerini yönetir. Aşağıdaki altı ekran, sistemin uçtan uca kullanıcı akışını (navigasyon hiyerarşisini) temsil eden düşük-orta çözünürlüklü (wireframe seviyesi) mock-up'lar olarak sunulmuştur: mutlu yolda S1 → S2 → S3 → S4, kullanıcı bir darboğazla karşılaştığında ise S4 → S5 akışı izlenir. Senaryo 4/5 ve R4/R15'te tanımlanan ağ doğrulama hatası için ise S3 sırasında devreye giren ayrı bir hata yolu tanımlanmıştır: S2 → S3 → S3b (Ağ Doğrulama Hatası), kullanıcı buradan "Haritaya Dön" ile S2'ye geri yönlendirilir.

Ekran 1 (S1) - Ana Harita ve Dashboard (Varsayılan Görünüm)



Ekran S1 — Ana Harita ve Dashboard (Varsayılan Görünüm)

Amacı: Kullanıcıyı sisteme ilk girdiğinde karşılayan, mevcut trafik durumunun kuşbakışı incelendiği ana çalışma masasıdır.

Görsel Hiyerarşi ve Düzen: * Ekranın sol %80'lik kısmı interaktif, WebGL destekli (Mapbox veya Leaflet) haritaya ayrılmıştır. Seçilen pilot bölgenin vektörel yol ağı bu alanda sergilenir. Sağ taraftaki %20'lik dikey panelde (Dashboard); sistem logoları, o an aktif olan senaryonun meta-verileri (Örn: "Base Scenario - Peak Hour 08:00") ve sistemin genel istatistiklerini (Toplam Araç Sayısı, Ağ Ortalaması Hız, Toplam Uzunluk) gösteren özet KPI (Temel Performans Göstergesi) kartları bulunur.

Etkileşim: Kullanıcı haritada serbestçe yakınlaştırma (zoom), uzaklaştırma ve kaydırma (pan) işlemleri yapabilir.

Ekran 2 (S2) - Müdahale Seçim Paneli



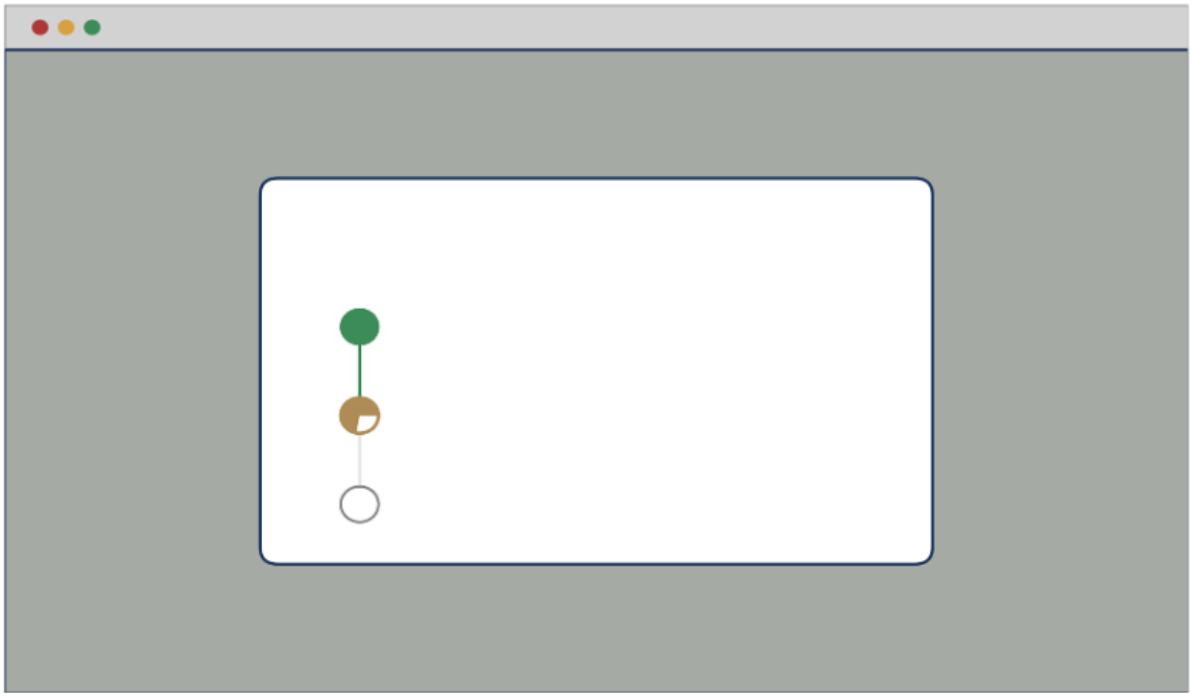
Ekran S2 — Müdahale Seçim Paneli (Sidebar Drawer)

Amacı: Kullanıcının spesifik bir yol veya kavşak üzerinde topolojik müdahale planladığı veri giriş ekranıdır.

Görsel Hiyerarşi ve Düzen: Kullanıcı harita üzerinde bir yola (Edge) veya kavşağa (Node) tıkladığında, odaklanmayı artırmak için harita hafifçe kararır (dimming efekti). Ekranın sağ tarafından (Dashboard'un üzerini örtecek şekilde) bir çekmece menü (Sidebar Drawer) kayarak açılır.

Etkileşim: Bu panel, sisteme tanımlı 6 ana kalemden oluşan "Müdahale Kataloğu"nu sunar. Butonlar sırasıyla şunlardır: **Şerit Kapat / Ekle, Tek Yöne Çevir, Yol Tipini / Hız Sınırını Değiştir, Hazır Kavşak Dönüşümü Uygula, Katılım Şeridi (Slip-Lane) Ekle, Bypass / Bağlantı Yolu Ekle.** Kullanıcının seçtiği müdahaleler, panelin alt kısmında bir e-ticaret "Alışveriş Sepeti (Shopping Cart)" mantığıyla alt alta birikir. Kullanıcı istediği zaman sepetten bir müdahaleyi çıkarabilir (undo) veya sağ üstteki "Analiz Et / Simüle Et" ana eylem (Call-to-Action) butonuna basarak süreci tetikleyebilir.

Ekran 3 (S3) - Yükleme ve Simülasyon Durumu Ekranı



Ekran S3 — Yükleme ve Simülasyon Durumu Ekranı (Loading Overlay / Stepper)

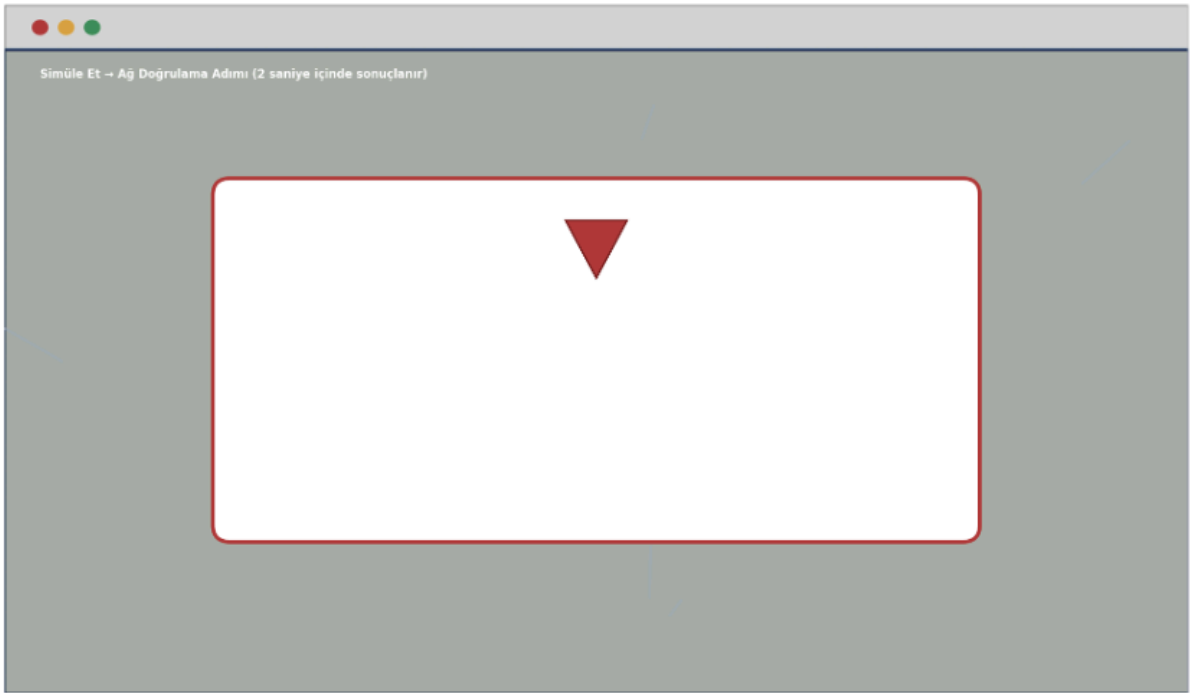
Amacı: Asenkron olarak arka planda koşturulan ve dakikalar sürebilecek simülasyon işlemi sırasında kullanıcıyı bilgilendirmek ve sistemin donmadığına dair feedback vermektir.

Görsel Hiyerarşi ve Düzen: Kullanıcı "Simüle Et" butonuna bastığında ekranın tam ortasında yarı saydam, asılı bir pencere (Modal) belirir. Bu pencerenin içinde ilerlemeyi gösteren bir adım çubuğu bulunur.

Etkileşim: Sistem, arka plandaki işlem durumuna (Polling üzerinden gelen API yanıtlarına) göre Stepper üzerindeki adımları günceller. Kullanıcı sırasıyla şu metinleri ve animasyonları görür:

1. *Ağ Doğrulanıyor (Yeşil Tik)*
2. *DUA Rotaları Hesaplanıyor (Dönen Spinner İşareti)*
3. *Sonuçlar İşleniyor...* Bu ekran, kullanıcının sistemi anlık olarak takip etmesini sağlarken, haritanın arka planda etkileşime kapatılmasını (blocking) önler (Opsiyonel olarak modal küçültülebilir).
4. *"Ağ Doğrulanıyor"* adımı başarısız olursa akış S4'e değil, aşağıdaki S3b hata ekranına yönlenir.

Ekran 3b (S3b) — Ağ Doğrulama Hatası (Hata Modalı)



Ekran S3b — Ağ Doğrulama Hatası (Hata Modalı)

Amacı: Senaryo 4 (İzole Düğüm Hatası) ile R4/R15 gereksinimlerinde tanımlanan güvenlik ağının arayüz karşılığını oluşturmaktır. Ağ doğrulama modülü bir hata tespit ettiğinde, simülasyonun çökmesini engellemek ve durumu kullanıcıya açıklamaktır.

Görsel Hiyerarşi ve Düzen: Simülasyon arka planda kuyruğa alınmadan önce, ekranın ortasında kırmızı çerçeveli ve dikkat çekici bir Hata Modalı belirir. Modalın içerisinde hatanın temel nedeni ("İzole Düğüm Tespit Edildi") açık ve net bir metinle vurgulanır.

Etkileşim: Geçersiz bir ağ tespit edildiğinde, bu ekran 2 saniye içinde arayüzde belirir. Kullanıcıya hatayı çözmesi için iki net eylem seçeneği sunulur: "Müdahaleyi Geri Al" ve "Haritaya Dön". Navigasyon hiyerarşisi S2 ↔ S3 ↔ S3b döngüsünü işleyerek, kullanıcı geçerli bir topoloji oluşturana kadar işlemi durdurur.

Ekran 4 (S4) - Karşılaştırmalı Sonuç Ekranı

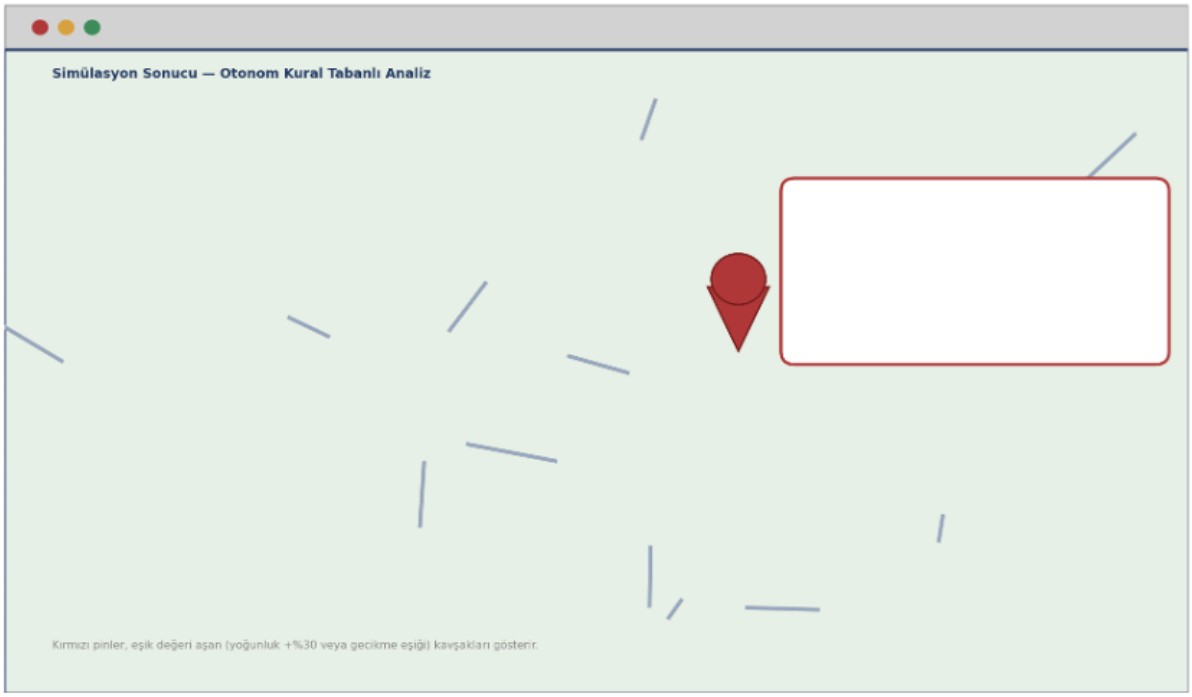


Amacı: Simülasyon tamamlandığında, yöneticilere sunulacak nihai karar destek verilerinin görselleştirildiği ana analiz ekranıdır.

Görsel Hiyerarşi ve Düzen: Ekranın merkezinde tek bir tam boy harita görünümü yer alır. Çift render yükünün tarayıcı performansını düşürmemesi adına bölünmüş ekran (split-screen) tasarımı tercih edilmemiştir. Yollar, o an ekranda aktif olan senaryonun trafik sıkışıklığına göre renk koduyla (Yeşil = Akıcı, Sarı = Yoğun, Koyu Kırmızı = Sıkışık) boyanır.

Etkileşim: Harita üzerinde yer alan bir geçiş butonu sayesinde kullanıcı; "Müdahale Öncesi (Base)" ve "Müdahale Sonrası" durumları arasında anında geçiş yapar. Ekranın alt kısmında yatay bir "Kazanım Tablosu" yer alır ve iki senaryo arasındaki makroskobik farkları özetler (Örn: "Delancey St. Kavşağı Gecikme: 45sn → 28sn (%38 İyileşme)"). Ayrıca bu panelde yer alan "Grafiklere git" butonu ile kullanıcı, yoğunluk ve gecikme değişimlerini günün saatlerine göre inceleyebileceği detaylı zaman serisi (time-series) grafiklerinin bulunduğu görünüme geçiş yapabilir.

Ekran 5 (S5) - Hata ve Kural Tabanlı Uyarı Modalı



Ekran S5 — Kural Tabanlı Uyarı Modalı (Warning Tooltip)

Amacı: Kullanıcının çözmeye çalıştığı problemin, sistemin farklı bir noktasında yeni bir darboğaz (bottleneck) yaratması durumunda sistemi otonom olarak uyarmasıdır.

Görsel Hiyerarşi ve Düzen: Simülasyon bittikten sonra harita üzerinde, olağan dışı yoğunluk artışı yaşayan kavşakların üzerinde kırmızı, dikkat çekici, animasyonlu pinler (marker) belirir.

Etkileşim: Kullanıcı bu kırmızı pine tıkladığında üzerinde küçük bir bilgi balonu (Tooltip) açılır. Balonun içerisinde kural tabanlı (rule-based) analiz sonucu üretilen spesifik bir metin yer alır: *"Uyarı: Bu müdahale, arka sokaktaki trafiği %45 oranında artırarak kapasite aşımına neden oldu."* Bu sayede kullanıcı, müdahalesinin kör noktalarını fark ederek senaryosunu revize etme şansı bulur.

5. SÖZLÜK

DUA (Dynamic User Assignment - Dinamik Kullanıcı Dengesi): Trafik mühendisliğinde Wardrop'un Birinci Prensibine[11] dayanan, araçların trafik ağı üzerindeki dağılımını modelleyen iteratif bir algoritmadır. Gerçek hayatta sürücülerin trafik yoğunluğuna göre alternatif rotalar araması davranışını simüle eder. Sistemde araçlar (ajanlar) önce en kısa mesafeli yola atanır; oluşan sanal tıkanıklıklar hafızaya alınarak simülasyon tekrar koşturulur. Bu döngü, hiçbir aracın rotasını değiştirerek seyahat süresini daha fazla kısaltamayacağı "Nash Dengesi" noktasına ulaşana kadar devam eder.

GEH İstatistiği: İngiliz trafik mühendisi Geoffrey E. Havers[6] tarafından geliştirilen, gözlemlenen gerçek trafik sayımları ile simülasyon modelinin ürettiği sentetik hacim tahminleri arasındaki farkı ölçen standart bir ampirik yaklaşımdır. Basit yüzdelik hata hesaplamalarının aksine, düşük hacimli yollardaki küçük sapmaların yaratacağı devasa yüzdelik hataları tolere edecek şekilde tasarlanmıştır.

Bu istatistiksel değerin 5.0'ın altında çıkması mükemmel, 10.0'ın altında kalması ise kabul edilebilir bir kalibrasyon eşiği olarak değerlendirilir.

KL Iraksama (Kullback-Leibler Divergence): Bilgi kuramında (information theory) bir olasılık dağılımının, ikinci bir referans olasılık dağılımından ne kadar farklı (uzak) olduğunu ölçen asimetrik bir göreceli entropi (relative entropy) metriğidir[8]. Projede, algoritmik olarak üretilen sentetik O-D seyahat taleplerinin, gerçek taksi verilerindeki[7] seyahat alışkanlıklarına (örüntülerine) ne kadar benzediğini ve sistemin ürettiği sentetik dağılımın gerçekliğe olan yakınlığını kanıtlamak için kullanılır.

OD Matrisi (Origin-Destination - Başlangıç-Hedef Matrisi): Ulaşım planlamasının en temel veri yapısıdır. Belirli bir coğrafi ağ üzerinde, bir zaman dilimi içerisinde nereden (başlangıç bölgesi) nereye (hedef bölgesi) ne kadar seyahat talebi olduğunu gösteren 2 boyutlu (N x N) bir karesel matristir. Satırlar çıkış noktalarını, sütunlar varış noktalarını, matrisin hücreleri (hücre değerleri) ise o iki nokta arasındaki toplam araç sayısını ifade eder. Sistem, bu makroskobik matrisi kullanarak mikroskobik araç rotaları üretir.

Edge / Node (Kenar / Düğüm): Sistemin altyapısını oluşturan Çizge Kuramı (Graph Theory) terimleridir.

- **Edge (Kenar):** Fiziksel yol segmentlerini (caddeler, sokaklar, otoyollar) temsil eder. Sistemin veri modelinde bir Edge; kapasite (şerit sayısı), uzunluk ve hız sınırı gibi fiziksel nitelikler taşır.
- **Node (Düğüm):** Bu yolların kesiştiği kavşak noktalarını, çıkmaz sokak uçlarını veya yol birleşim yerlerini ifade eder. Düğümler geçiş hakkı kurallarını ve sinyalizasyon (trafik ışığı) mantığını barındırır.

İzole Düğüm: Topolojik bir hatayı tanımlar. Kullanıcının yaptığı yanlış bir yol kapatma müdahalesi (örn: tek yönlü yolların ters bağlanması) sonucunda, haritanın geri kalanıyla bağlantısı kesilen ve sisteme giriş/çıkış yapılamayan düğüm veya düğüm gruplarıdır. İzole düğümler, en kısa yol algoritmalarının sonsuz döngüye girmesine veya çözüm bulamamasına neden olduğu için simülasyon motorunu işlemez hale getiren kritik (fatal) hatalardır.

netconvert: SUMO simülasyon motorunun ağ dönüştürme ve işleme aracıdır. OpenStreetMap gibi dış kaynaklardan alınan harita verilerini sistemin anlayabileceği matematiksel ağ dosyalarına (.net.xml) çevirir. Aynı zamanda yol kapatma, şerit ekleme gibi geometrik müdahalelerin arka planda uygulandığı ve ağdaki topolojik hataların tespit edildiği çekirdek yazılım bileşenidir.

od2trips: Makroskobik O-D (Başlangıç-Hedef) matrislerini, simülasyonda kullanılacak mikroskobik araç seyahatlerine (trip) dönüştüren SUMO yardımcı aracıdır. Bölgesel yüzdelik seyahat taleplerini, tekil araçların başlangıç ve bitiş yolları olarak somutlaştırır.

duarouter: Sistemdeki her bir araç için harita üzerindeki en kısa veya en optimum rotayı hesaplayan SUMO yönlendirme aracıdır. Araçların seyahat planlarını alarak onlara adım adım sokak yönlendirmelerini içeren rota dosyalarını (.rou.xml) üretir ve DUA iterasyonlarında kilit rol oynar.

fcd-output.xml (Floating Car Data): SUMO tarafından üretilen ve simülasyondaki her bir aracın her bir saniyedeki anlık konumunu (koordinat), hızını ve bekleme durumunu listeleyen son derece detaylı (mikroskobik) ham sonuç (log) dosyasıdır.

edgedata-output.xml: Simülasyon sonucunda araçların tek tek pozisyonları yerine, yolların geneli hakkında bilgi veren özet (makroskobik) sonuç dosyasıdır. Belirli bir zaman dilimi içinde bir yol segmentinden (edge) geçen toplam araç sayısını, ortalama trafik yoğunluğunu ve bekleme süresini içerir. Karar destek sistemindeki ısı haritaları ve metrik tabloları doğrudan bu dosya ayrıştırılarak (parse edilerek) oluşturulur.

6. KAYNAKLAR

[1] PTV Group, "PTV Vissim User Manual," Karlsruhe, Germany. [Çevrimiçi]. Erişim: https://www.ptvgroup.com

[2] P. A. Lopez *et al.*, "Microscopic traffic simulation using SUMO," in *Proc. 21st Int. Conf. Intell. Transp. Syst. (ITSC)*, Maui, HI, USA, 2018, pp. 2575-2582.

[3] A. Horni, K. Nagel, and K. W. Axhausen, Eds., *The Multi-Agent Transport Simulation MATSim*. London, U.K.: Ubiquity Press, 2016.

[4] OpenStreetMap contributors, "OpenStreetMap," 2025. [Çevrimiçi]. Erişim: https://www.openstreetmap.org

[5] A. G. Wilson, "A statistical theory of spatial distribution models," *Transp. Res.*, vol. 1, no. 3, pp. 253-269, 1967.

[6] Federal Highway Administration, "Traffic Analysis Toolbox Volume III: Guidelines for Applying Traffic Microsimulation Modeling Software," U.S. Dept. Transp., Washington, DC, USA, Rep. FHWA-HRT-04-040, 2004.

[7] New York City Taxi and Limousine Commission, "TLC Trip Record Data," 2024. [Çevrimiçi]. Erişim: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[8] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Stat.*, vol. 22, no. 1, pp. 79-86, 1951.

[9] S. Ramírez, "FastAPI," 2018. [Çevrimiçi]. Erişim: https://fastapi.tiangolo.com/

[10] Meta Platforms, Inc., "React," 2013. [Çevrimiçi]. Erişim: https://react.dev/

[11] J. G. Wardrop, "Some theoretical aspects of road traffic research," *Proc. Inst. Civ. Eng.*, vol. 1, no. 3, pp. 325-362, 1952.

7. ARAÇLAR VE METODOLOJİ

Bu dokümanın hazırlanma sürecinde (fikir geliştirme, metin/tablo/diyagram/resim düzenleme ve 4. ve 5. bölümlerin ilk taslakları) ChatGPT, Claude ve Gemini araçlarından destek alınmıştır. Ancak tüm teknik kararlar, sistem mimarisi ve nihai kontroller proje ekibi tarafından yapılmış olup, içeriğin doğruluğu, özgünlüğü ve teknik yeterliliği konusundaki tüm sorumluluk ekibimize aittir.